

LAPORAN TEKNIS DATA SCIENTIST

DiagnoKu: *Pre-screening System*



Disusun Oleh:

Tim Data Scientist (CC26-PSU200)

Dian Ayu Azizah : CDCC290D6X2224

Muhammad Irfan Wahyudi : CDCC180D6Y1802

Tema:

Healty Lives and Well-being

CAPSTONE PROJECT
CODING CAMP 2026
DICODING X DBS FOUNDATION

DAFTAR ISI

DAFTAR ISI	2
BAB I PENDAHULUAN & RINGKASAN EKSEKUTIF	4
1.1. Latar Belakang.....	4
1.2. Tujuan Utama Pipeline	4
BAB II ARSITEKTUR PIPELINE DATA.....	5
2.1. Setup dan Import Library	5
2.1.1. Import Library & Konfigurasi Global Environment	5
2.1.2. Mount Google Drive	5
2.1.3. Load Dataset & Downcasting Tipe Data	6
2.2. Assessing Data & Exploratory Data Analysis (EDA).....	6
2.2.1. Fungsi Deep Assessing Report	6
2.2.2. Analisis Distribusi Frekuensi Target (Awal).....	7
2.2.3. Distribusi Kerapatan Gejala per Sampel (Awal)	8
2.2.4. Formulasi Pertanyaan Bisnis dan Hipotesis.....	10
2.3. Cleaning Data	10
2.3.1. Eliminasi Baris Duplikat.....	11
2.3.2. Identifikasi Penyakit Minor (<50).....	11
2.3.3. Bedah Statistik Visual Penyakit Minor	12
2.3.4. Eksekusi Pemangkasan Penyakit Minor	14
2.3.5. Pemetaan Kerapatan Gejala Pasca Pemangkasan Penyakit	14
2.3.6. Assessment Penilaian Risiko Sub-Standard Gejala (<3).....	15
2.3.7. Isolasi Subset Partisi Data.....	16
2.3.8. Implementasi Boolean Max OR Logic Aggregation	16
2.3.9. Penyaringan Hasil Agregasi Kompresi.....	17
2.3.10. Rekonsolidasi & Integrasi Data Bersih	17
2.3.11. Verifikasi dan Audit Akhir	18
2.4. Explanatory Analysis (Post-Cleaning Validation)	18

2.4.1.	Distribusi Frekuensi Target Penyakit (Final)	18
2.4.2.	Analisis Kerapatan Gejala per Baris (Final).....	19
2.4.3.	Audit Evaluasi Pertanyaan Bisnis	21
2.4.4.	Konsolidasi Metrik Penyelusuran Perubahan Data.....	22
2.4.5.	Rekayasa Defensif & Pembersihan Sekunder (Validasi Akhir)	23
2.4.6.	Ekspor Data Bersih Akhir	23
2.5.	Pembuatan Kamus Sinonim Gejala	24
2.5.1.	Re-load Dataset Bersih	24
2.5.2.	Ekstraksi dan Pengurutan Fitur Gejala	24
2.5.3.	Pemetaan Manual Kmaus Relasional Sinonim Gejala Indonesia.....	25
2.5.4.	Ekspor Kamus Sinonim Gejala.....	25
2.6.	Pembuatan Kamus Penyakit (Otomatisasi Translasi Target).....	25
2.6.1.	Instalasi Library Deep Translator	25
2.6.2.	Ekstraksi Larik Nama Penyakit Unik.....	26
2.6.3.	Instalasi Engine Penerjemah & Smoke Testing.....	26
2.6.4.	Eksekusi Penerjemahan Massal (Automated Batch Translation).....	26
2.6.5.	Inspeksi Manual Sampel Hasil Translasi.....	27
2.6.6.	Penyimpanan Hasil Pemetaan Kamus Penyakit	27
2.7.	Catatan Integrasi Dashboard (Tahap Lanjutan)	28
BAB III KESIMPULAN & REKOMENDASI ACTION ITEM		30
3.1.	Kesimpulan Analisis Data	30
3.2.	Rekomendasi Action Item	30

BAB I

PENDAHULUAN & RINGKASAN EKSEKUTIF

1.1. Latar Belakang

Kendala utama dalam membangun model klasifikasi medis yang kokoh seringkali bersumber dari kualitas data mentah yang dikumpulkan dari sumber *public*. Data mentah tersebut umumnya memiliki karakteristik:

1. Ketimpangan kelas (*class imbalance*) yang ekstrem akibat perbedaan prevalensi penyakit di dunia nyata.
2. Redundansi informasi akibat adanya duplikasi data pasien.
3. *Random noise* berupa rekam medis yang tidak valid atau kekurangan informasi (memiliki jumlah gejala yang terlalu sedikit untuk diagnose hipotesis)>

Laporan teknis ini mendokumentasikan seluruh rangkaian arsitektur *data engineering* dan *data wrangling pipeline* yang diterapkan pada 'Disease and symptoms dataset.csv'. Rangkaian proses ini dirancang untuk mengubah data mentah biner berskala besar menjadi dataset siap latih berkode referensi 'diagnoku_biner_v5.csv'.

Selain pembersihan data, laporan ini juga memuat tahap standarisasi bahasa melalui pembuatan repositori metadata format JSON untuk menjembatani kesenjangan terminology medis klinis (Bahasa Inggris) dengan input natiral pengguna (Bahasa Indonesia).

1.2. Tujuan Utama Pipeline

1. Mengurnagi beban memori RAM lingkungan kerja (Google Colab).
2. Mengeliminasi pencilan (*outliers*) sampel nouse tanpa kehilangan variasi penyakit asli secara masif.
3. Mengembangkan logika agregasi berbasis *Boolean Logic* untuk menyelamatkan sampel minim informasi.
4. Mmembangun *automated machine translation engine* dan repositori sinonim kamus pemetaan gejala dan penyakit untuk kebutuhan *deployment* sistem Diagnoku.

BAB II

ARSITEKTUR PIPELINE DATA

2.1. Setup dan Import Library

Pada tahap inisiasi, dilakukan penyusunan infrastruktur lingkungan kerja (*environment*). Kebutuhan Pustaka modular dikategorikan ke dalam empat faset utama: manipulasi data, visualisasi, integrasi cloud storage, dan penanganan format serialisasi objek.

2.1.1. Import Library & Konfigurasi Global Environment

```
# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Google Drive integration
from google.colab import drive

# JSON for dictionary export
import json

# Set style untuk visualisasi yang lebih baik
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
sns.set_context("notebook", font_scale=1.2)

# Untuk menampilkan semua kolom saat display
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 50)
```

Pustaka 'pandas' dan 'numpy' merupakan standar industri de-facto untuk operasi vektorisasi dan manipulasi array multidimensi. Konfigurasi 'pd.set_option' diterapkan untuk mencegah pemotongan visualisasi matriks berdimensi tinggi saat inspeksi manual dilakukan. Pengaturan style 'seaborn-v0_8-whitegrid' dipanggil guna menjamin grafik yang dihasilkan memenuhi kaidah standarisasi laporan ilmiah dengan kontras warna tinggi dan keterbacaan grid yang optimal.

2.1.2. Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Integrasi penyimpanan berbasis cloud Google Drive ke dalam sistem berkas lokal mesin virtual Google Colab bertujuan untuk membentuk data *pipeline stream* yang aman. Hal ini menghindari proses unggah manual

berulang kali yang rawan interupsi jaringan dan menghemat ruang penyimpanan lokal *disk ephemeral runtime* Colab.

2.1.3. Load Dataset & Downcasting Tipe Data

```
df_raw = pd.read_csv('/content/drive/MyDrive/DS_CC26-PSU200/Raw
Dataset/Disease and symptoms dataset.csv')
# Optimasi tipe data int8 untuk efisiensi memori (Data Biner)
cols_biner = df_raw.columns.drop('diseases')
df_raw[cols_biner] = df_raw[cols_biner].apply(pd.to_numeric,
downcast='integer')

print(f"\nDataset Awal : {df_raw.shape} | Memory:
{df_raw.memory_usage().sum()/1024**2:.2f} MB")
display(df_raw.head())
```

Secara default, fungsi 'pd.read_csv' akan mengalokasikan memori berupa tipe data 'int64' untuk kolom bertipe numerik bulat. Mengingat dataset ini merupakan Matriks Spasial Biner (bernilai 0 atau 1), penggunaan 'int64' sangat tidak efisien. Operasi *downcasting* 'pd.to_numeric(..., downcast='integer') akan memaksa pandas menurunkan tipe data ke batas bit terendah yang mampu menampung nilai tersebut, yaitu 'int8'. Langkah ini secara drastic memangkas konsumsi RAM menjadi hanya 90.67 MB, mencegah terjadinya *Out-Of-Memory* (OOM) error pada pemrosesan tahap berikutnya.

2.2. Assessing Data & Exploratory Data Analysis (EDA)

Sebelum melakukan tindakan manipulasi data dilakukan audit untuk mendeteksi anomali struktural dan statistic dalam dataset.

2.2.1. Fungsi Deep Assessing Report

```
def deep_assessing(df, name="Dataset"):
    print(f"=== ASSESSING REPORT: {name} ===")
    print(f"1. Bentuk Data (Shape): {df.shape}")
    print(f"2. Total Duplikat: {df.duplicated().sum()} baris")
    print(f"3. Total Missing Values: {df.isnull().sum().sum()} sel")

    # Cek tipe data
    tipe_data = df.dtypes.value_counts()
    print(f"4. Tipe Data:\n{tipe_data.to_string()}")

    # Cek baris kosong gejala
    symptom_cols = df.columns.drop('diseases', errors='ignore')
    zero_symptom_rows = (df[symptom_cols].sum(axis=1) == 0).sum()
    print(f"5. Baris dengan 0 Gejala: {zero_symptom_rows}")
    print(f"6. Jumlah Penyakit Unik: {df['diseases'].nunique()}")
    print("-" * 35)

deep_assessing(df_raw, "Dataset Awal")
```

Output:

```
=== ASSESSING REPORT: Dataset Awal ===
1. Bentuk Data (Shape): (246945, 378)
2. Total Duplikat: 57298 baris
3. Total Missing Values: 0 sel
4. Tipe Data: int8 377 object 1
5. Baris dengan 0 Gejala: 0
6. Jumlah Penyakit Unik: 773
-----
```

Fungsi modular 'deep_assessing' bertindak sebagai gerbang kendali mutu (*quality gate control*). Hasil log menunjukkan penemuan kritis, terdapat 57.298 baris duplikat yang mengindikasikan tingginya rekaman berulang. Sisi positifnya, data bersih dari missing values (0 sel kosong), dan tidak ditemukan baris berstatus "0 Gejala", yang berarti seluruh baris rekam medis mengindikasikan minimal satu gejala aktif.

2.2.2. Analisis Distribusi Frekuensi Target (Awal)

```
# Top 20 penyakit terbanyak
disease_counts = df_raw['diseases'].value_counts()
top20_diseases = disease_counts.head(20)

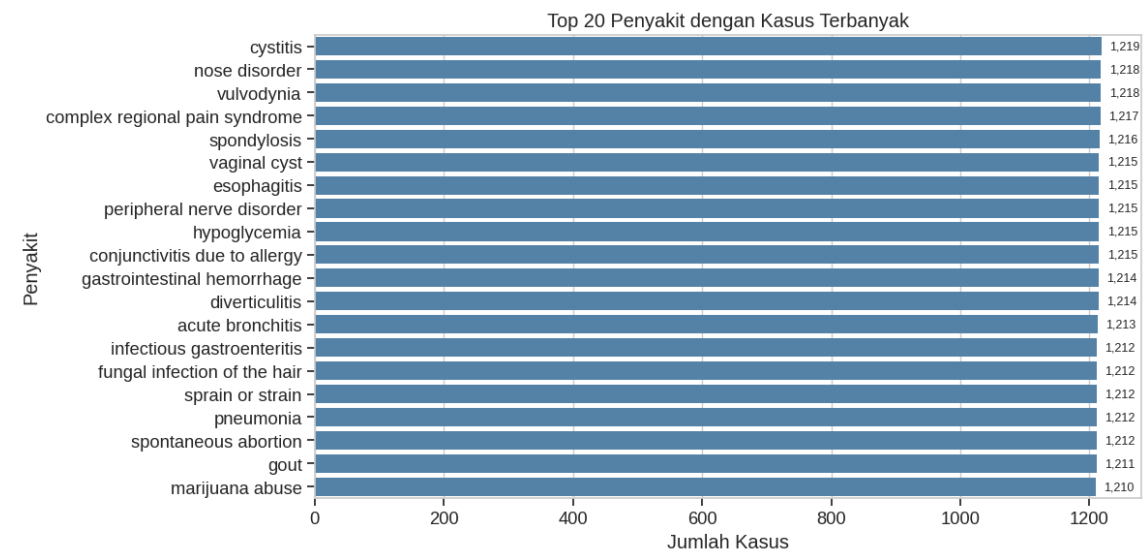
plt.figure(figsize=(12, 6))
ax = sns.barplot(x=top20_diseases.values, y=top20_diseases.index,
color='steelblue')

for i, (nilai, nama) in enumerate(zip(top20_diseases.values,
top20_diseases.index)):
    ax.text(nilai + (nilai * 0.01), i, f'{nilai:,}',
        va='center', ha='left', fontsize=9)

plt.xlabel('Jumlah Kasus')
plt.ylabel('Penyakit')
plt.title('Top 20 Penyakit dengan Kasus Terbanyak')
plt.tight_layout()
plt.show()

print(f"\nTotal penyakit unik di awal: {len(disease_counts)}")
print(f"Penyakit dengan kasus terbanyak: '{disease_counts.index[0]}'
({disease_counts.iloc[0]:,} kasus)")
print(f"Penyakit dengan kasus tersedikit: '{disease_counts.index[-1]}'
(1 kasus)")
```

Output:



Total penyakit unik di awal: 773

Penyakit dengan kasus terbanyak: 'cystitis' (1,219 kasus)

Penyakit dengan kasus tersedikit: 'kaposi sarcoma' (1 kasus)

Grafik ini menyingkap fenomena ketimpangan kelas yang ekstrem (*severe class imbalance*). Kondisi ini menciptakan tantangan bagi model *supervised learning* karena algoritma klasifikasi (seperti Decision Trees atau Naive Bayes) cenderung mengalami bias yang kuat ke arah kelas mayoritas dan gagal mempelajari pola esensial dari kelas minoritas yang kekurangan representasi sampel.

2.2.3. Distribusi Kerapatan Gejala per Sampel (Awal)

```
# Hitung jumlah gejala per baris di data awal
gejala_cols_awal = df_raw.columns.drop('diseases')
n_gejala_awal = df_raw[gejala_cols_awal].sum(axis=1)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(n_gejala_awal, bins=20, color='steelblue') # hapus
edgecolor
plt.xlabel('Jumlah Gejala per Baris')
plt.ylabel('Frekuensi')
plt.title('Distribusi Jumlah Gejala (Data Awal)')

# Tambahkan label nilai pada batang tertinggi untuk konteks
max_freq = n_gejala_awal.value_counts().sort_index().max()
max_bin = n_gejala_awal.value_counts().sort_index().idxmax()
plt.text(max_bin + 0.2, max_freq, f'Mode: {max_bin} gejala',
va='bottom', fontsize=9, color='gray')
```



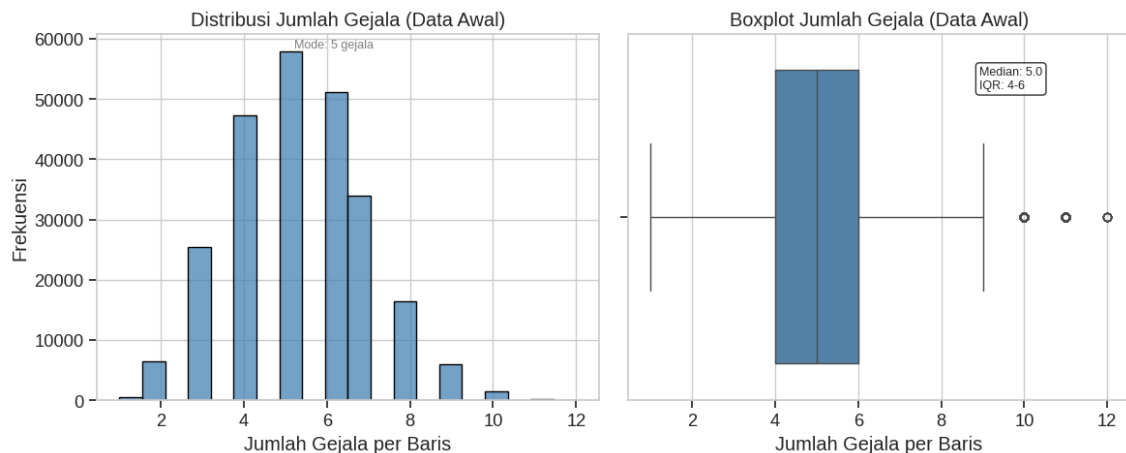
```
plt.subplot(1, 2, 2)
sns.boxplot(x=n_gejala_awal, color='steelblue') # warna konsisten
plt.xlabel('Jumlah Gejala per Baris')
plt.title('Boxplot Jumlah Gejala (Data Awal)')

# Tambahkan statistik ringkas pada plot
median_val = n_gejala_awal.median()
q1_val = n_gejala_awal.quantile(0.25)
q3_val = n_gejala_awal.quantile(0.75)
plt.annotate(f'Median: {median_val}\nIQR: {q1_val:.0f}-{q3_val:.0f}',
             xy=(0.7, 0.85), xycoords='axes fraction', fontsize=9,
             bbox=dict(boxstyle='round', facecolor='white',
alpha=0.8))

plt.tight_layout()
plt.show()

print("Statistik jumlah gejala per baris (Data Awal):")
print(n_gejala_awal.describe())
print(f"\nPersentase baris dengan gejala <3: {(n_gejala_awal <
3).mean() * 100:.2f}%")
```

Output:



Statistik jumlah gejala per baris (Data Awal):

```
count    246945.000000
mean         5.332851
std         1.640610
min          1.000000
25%          4.000000
50%          5.000000
75%          6.000000
max         12.000000
dtype: float64
```

Persentase baris dengan gejala <3: 2.79%

Analisis statistik ini mendasari penetapan batas ambang klinis (clinical threshold pruning). Dalam praktiknya, sebuah penegakan diagnosa diferensial

yang akurat sulit dilakukan apabila data rekam medis pasien hanya mencantumkan 1 atau 2 gejala umum (misal: hanya "demam" tanpa indikator pendukung). Oleh karena itu, porsi data 2.79% ini diklasifikasikan sebagai sub-standard rekam medis yang harus ditangani pada fase pembersihan agar tidak menurunkan performa metrik spesifisitas mode.

2.2.4. Formulasi Pertanyaan Bisnis dan Hipotesis

```
print("="*60)
print("PERTANYAAN BISNIS YANG DAPAT DIUKUR")
print("="*60)
questions = [
    "1. Berapa banyak penyakit langka (<50 kasus) yang perlu dihapus?",
    "2. Bagaimana distribusi jumlah gejala per penyakit?",
    "3. Penyakit apa yang memiliki pola gejala paling unik (sedikit overlap)?",
    "4. Gejala apa yang paling sering muncul dan bersifat general?",
    "5. Setelah cleaning, berapa penyakit yang memiliki minimal 3 gejala?"
]
for q in questions: print(q)

print("\n" + "="*60)
print("HIPOTESIS AWAL")
print("="*60)
print("- Data memiliki banyak penyakit minor yang akan mengganggu performa model")
print("- Sebagian kecil baris memiliki gejala <3 yang perlu ditangani")
print("- Setelah cleaning, dataset akan lebih seimbang dan siap untuk modeling")
```

Output:

```
=====
PERTANYAAN BISNIS YANG DAPAT DIUKUR
=====
1. Berapa banyak penyakit langka (<50 kasus) yang perlu dihapus?
2. Bagaimana distribusi jumlah gejala per penyakit?
3. Penyakit apa yang memiliki pola gejala paling unik (sedikit overlap)?
4. Gejala apa yang paling sering muncul dan bersifat general?
5. Setelah cleaning, berapa penyakit yang memiliki minimal 3 gejala?

=====
HIPOTESIS AWAL
=====
- Data memiliki banyak penyakit minor yang akan mengganggu performa model
- Sebagian kecil baris memiliki gejala <3 yang perlu ditangani
- Setelah cleaning, dataset akan lebih seimbang dan siap untuk modeling
```

Formulasi ini berfungsi menetapkan tujuan analitis yang objektif untuk mengukur efektivitas *data cleaning pipeline* berdasarkan metrik kuantitatif.

2.3. Cleaning Data

Fase ini merupakan inti dari proses rekayasa data untuk menyaring dan membersihkan noise struktural berdasarkan parameter batasan yang telah ditentukan.

2.3.1. Eliminasi Baris Duplikat

```
# Hapus semua baris duplikat
df_clean = df_raw.drop_duplicates()

print(f"Jumlah baris sebelum: {len(df_raw):,}")
print(f"Jumlah baris setelah: {len(df_clean):,}")
print(f"Jumlah duplikat yang dihapus: {len(df_raw) -
len(df_clean):,}")
```

Output:

```
Jumlah baris sebelum: 246,945
Jumlah baris setelah: 189,647
Jumlah duplikat yang dihapus: 57,298
```

Penghapusan duplikat secara absolut sangat krusial untuk mencegah terjadinya fenomena *data leakage* (kebocoran data) saat pembagian partisi subset *training* dan *testing* dilakukan pada fase pemodelan. Jika baris identik dibiarkan menyebar, performa pengujian model akan mengalami estimasi optimis yang semu.

2.3.2. Identifikasi Penyakit Minor (<50)

```
# Tampilkan penyakit dengan frekuensi <50
rare_before =
df_clean['diseases'].value_counts()[df_clean['diseases'].value_counts(
) < 50]
print(f"Jumlah penyakit dengan kasus <50: {len(rare_before)}")
print("\nContoh 10 penyakit minor:")
print(rare_before.head(10))
```

Output:

```
Jumlah penyakit dengan kasus <50: 360
```

```
Contoh 10 penyakit minor:
```

diseases	
thoracic outlet syndrome	48
retinopathy due to high blood pressure	47
cerebral palsy	47
chlamydia	47
lung cancer	47
peripheral arterial embolism	47
polymyalgia rheumatica	47
systemic lupus erythematosus (sle)	46
poisoning due to sedatives	46
foreign body in the gastrointestinal tract	45

```
Name: count, dtype: int64
```

Keberadaan 360 penyakit minor ini mewakili 46.6% dari total variasi penyakit awal (360 dari 773). Kelas-kelas ini kekurangan massa data kritis (*critical mass of data*) yang dibutuhkan oleh algoritma klasifikasi untuk mengekstrak batas keputusan statistik (*decision boundary*) yang reliabel.

2.3.3. Bedah Statistik Visual Penyakit Minor

```
# Kombinasi: tabel detail + histogram ringkasan
disease_counts_clean = df_clean['diseases'].value_counts()
low_freq = disease_counts_clean[disease_counts_clean < 50]

# Hitung distribusi per nilai unik untuk tabel
low_freq_counts = low_freq.value_counts().sort_index()

# Tampilkan tabel terlebih dahulu
print("\nDetail Distribusi Penyakit dengan Kasus <50:")
print("-" * 40)
print(f"{'Frekuensi Kasus':<20} {'Jumlah Penyakit':<15}")
print("-" * 40)
for freq, count in low_freq_counts.items():
    print(f"{freq:<20} {count:<15}")
print("-" * 40)
print(f"Total: {len(low_freq)} penyakit")

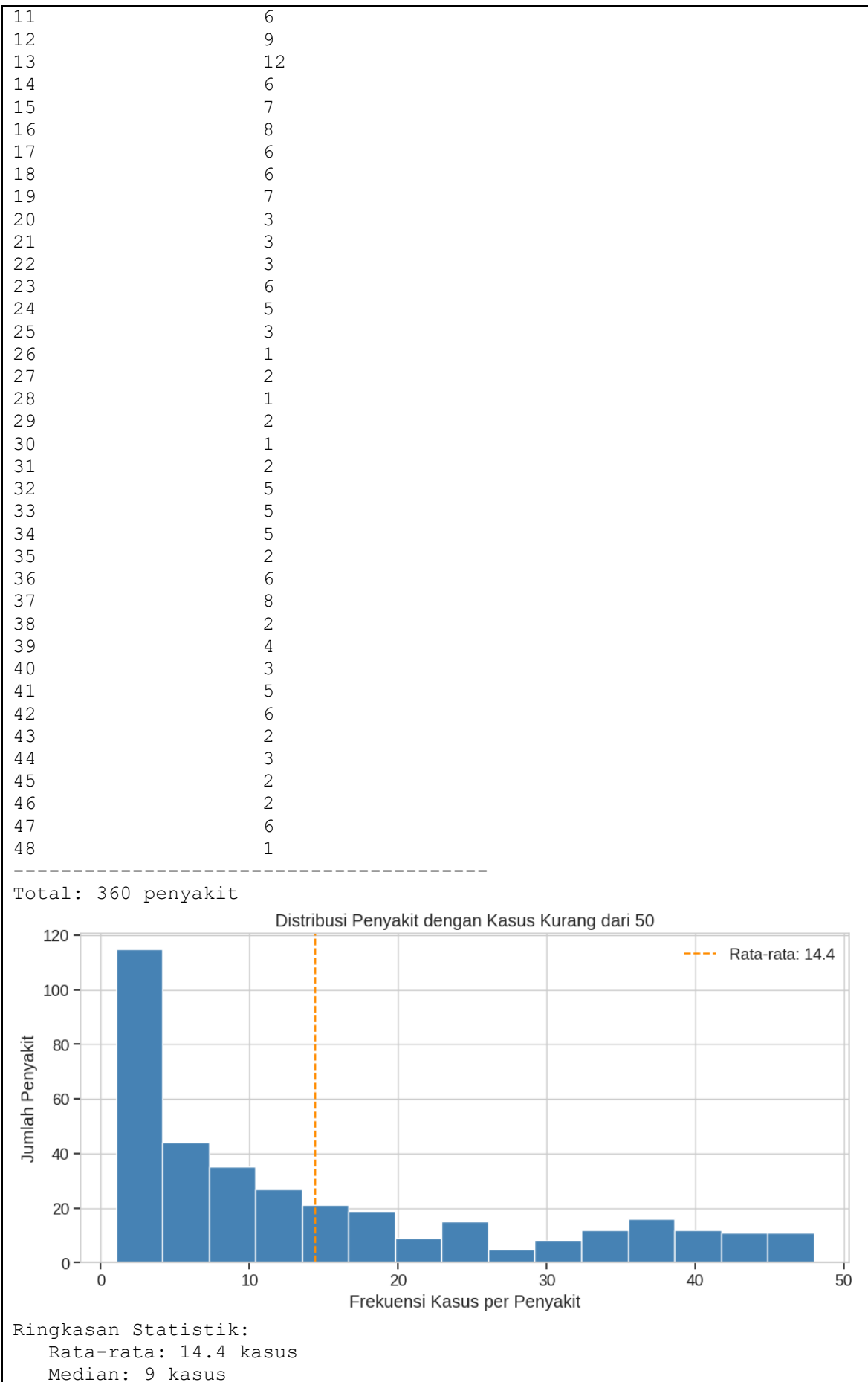
# Kemudian histogram untuk visualisasi pola
plt.figure(figsize=(10, 5))
plt.hist(low_freq, bins=15, color='steelblue', edgecolor='white')
plt.xlabel('Frekuensi Kasus per Penyakit')
plt.ylabel('Jumlah Penyakit')
plt.title('Distribusi Penyakit dengan Kasus Kurang dari 50')
plt.axvline(x=low_freq.mean(), color='darkorange', linestyle='--',
            label=f'Rata-rata: {low_freq.mean():.1f}')
plt.legend(frameon=False)
plt.tight_layout()
plt.show()

print(f"\nRingkasan Statistik:")
print(f"    Rata-rata: {low_freq.mean():.1f} kasus")
print(f"    Median: {low_freq.median():.0f} kasus")
print(f"    Min: {low_freq.min()} kasus")
print(f"    Max: {low_freq.max()} kasus")
```

Output:

Detail Distribusi Penyakit dengan Kasus <50:

Frekuensi Kasus	Jumlah Penyakit
1	46
2	29
3	26
4	14
5	11
6	20
7	13
8	20
9	7
10	8



Min: 1 kasus
Max: 48 kasus

Distribusi condong positif (*positively skewed*). Tercatat 46 penyakit hanya memiliki 1 sampel tunggal di seluruh dataset. Rentang nilai tengah (Median) berada pada 9 kasus.

Pola sebaran long-tail distribution ini membuktikan bahwa keberadaan penyakit langka ini bersifat merusak bagi model klasifikasi multi-kelas (*multi-class classifier*). Sampel dengan frekuensi 1 s.d 5 tidak membawa informasi variasi kombinasi gejala yang memadai, melainkan berperan sebagai pencilan acak (*stochastic outliers*)

2.3.4. Eksekusi Pemangkasan Penyakit Minor

```
# Filter dan assessment
rare_diseases =
df_clean['diseases'].value_counts()[df_clean['diseases'].value_counts(
) < 50].index

print(f"Sebelum: {len(df_clean)} baris,
{df_clean['diseases'].nunique()} penyakit")
print(f"Menghapus {len(rare_diseases)} penyakit langka (total
{(df_clean['diseases'].isin(rare_diseases)).sum():,} baris)")

df_clean = df_clean[~df_clean['diseases'].isin(rare_diseases)]

print(f"Sesudah: {len(df_clean)} baris,
{df_clean['diseases'].nunique()} penyakit")
print(f"Min sampel sekarang:
{df_clean['diseases'].value_counts().min()}")
```

Output:

```
Sebelum: 189647 baris, 773 penyakit
Menghapus 360 penyakit langka (total 5,193 baris)
Sesudah: 184454 baris, 413 penyakit
Min sampel sekarang: 50
```

Menggunakan operator negasi bitwise (~) dan dipadukan dengan metode '.isin()', pipeline berhasil memangkas 360 target penyakit tak layak latih. Melalui langkah ini, kami mengorbankan sebagian kecil baris data (5.193 baris atau 2.7%) untuk meningkatkan stabilitas serta keandalan sisa 413 kelas utama secara signifikan.

2.3.5. Pemetaan Kerapatan Gejala Pasca Pemangkasan Penyakit

```
gejala_columns = [col for col in df_clean.columns if col != 'diseases']
print(f"Jumlah kolom gejala: {len(gejala_columns)}")

df_clean = df_clean.copy()
```

```
df_clean['n_gejala'] = df_clean[gejala_columns].sum(axis=1)

print("\nStatistik jumlah gejala per baris (setelah hapus penyakit
minor):")
print(df_clean['n_gejala'].describe())
```

Output:

```
Jumlah kolom gejala: 377

Statistik jumlah gejala per baris (setelah hapus penyakit minor):
count      184454.000000
mean         5.470047
std          1.652181
min           1.000000
25%           4.000000
50%           5.000000
75%           7.000000
max          12.000000
Name: n_gejala, dtype: float64
```

Terisolasi 377 kolom gejala biner. Nilai rata-rata indikasi gejala bergeser naik menjadi 5.47 gejala, namun nilai minimum ekstrem tetap bertahan pada level 1 gejala.

2.3.6. Assessment Penilaian Risiko Sub-Standard Gejala (<3)

```
baris_kurang_gejala = df_clean[df_clean['n_gejala'] < 3]

print("="*50)
print("ASSESSMENT SEBELUM PENGGABUNGAN")
print("="*50)
print(f"Total baris: {len(df_clean):,}")
print(f"Baris dengan gejala >=3: {len(df_clean) -
len(baris_kurang_gejala):,}")
print(f"Baris dengan gejala <3: {len(baris_kurang_gejala):,}")
print(f"Persentase baris <3:
{len(baris_kurang_gejala)/len(df_clean)*100:.2f}%")
print(f"\nJumlah penyakit unik pada baris <3:
{baris_kurang_gejala['diseases'].nunique()}")
```

Output:

```
=====
ASSESSMENT SEBELUM PENGGABUNGAN
=====

Total baris: 184,454
Baris dengan gejala >=3: 180,121
Baris dengan gejala <3: 4,333
Persentase baris <3: 2.35%

Jumlah penyakit unik pada baris <3: 409
```

Angka 409 penyakit unik membuktikan bahwa masalah minimnya gejala (<3) bukanlah karakteristik unik dari beberapa penyakit tertentu. Isu ini merupakan indikasi adanya penyebaran *nosie* acak yang merata di hampir seluruh kelas penyakit. Pembuangan secara langsung terhadap 4.333 baris ini berisiko mendegradasi keutuhan sebaran frekuensi data pada 409 penyakit

tersebut. Oleh karena itu, diperlukan strategi penyelamatan data (*data recovery strategy*).

2.3.7. Isolasi Subset Partisi Data

```
df_ok = df_clean[df_clean['n_gejala'] >= 3].copy()
df_to_merge = df_clean[df_clean['n_gejala'] < 3].copy()

df_ok = df_ok.drop(columns=['n_gejala'])
df_to_merge = df_to_merge.drop(columns=['n_gejala'])

print(f>Data OK (>=3 gejala): {len(df_ok):,} baris")
print(f>Data perlu digabung (<3 gejala): {len(df_to_merge):,} baris")
```

Output:

```
Data OK (>=3 gejala): 180,121 baris
Data perlu digabung (<3 gejala): 4,333 baris
```

Melakukan pemisahan fisik via `.copy()` memecah dataset menjadi dua entitas runtime mandiri: `df_ok` (180.121 baris data berkualitas tinggi) dan `df_to_merge` (4.333 baris data sub-standard), untuk mencegah terjadinya error `SettingWithCopyWarning` di Pandas.

2.3.8. Implementasi Boolean Max OR Logic Aggregation

```
# Gabungkan baris dengan penyakit yang sama menggunakan max (OR logic)
df_merged = df_to_merge.groupby('diseases')[gejala_columns].max()
df_merged['n_gejala'] = df_merged[gejala_columns].sum(axis=1)

print(f>Hasil penggabungan: {len(df_merged)} penyakit")
print("\nDistribusi jumlah gejala setelah penggabungan:")
print(df_merged['n_gejala'].value_counts().sort_index())
```

Output:

```
Hasil penggabungan: 409 penyakit

Distribusi jumlah gejala setelah penggabungan:
n_gejala
2      11
3       7
4      15
5      13
6      32
7      79
8      83
9      62
10     72
11     26
12      9
Name: count, dtype: int64
```

Operasi `'groupby('diseases').max()'` di atas mengimplementasikan skema akumulasi pengetahuan berbasis Logika OR Biner. Untuk penyakit D_k , jika terdapat beberapa rekam medis yang mengalami fragmentasi informasi (misal: Rekam medis A hanya mencatat gejala S_1 , Rekam medis B hanya

mencatat gejala S₂), maka penggabungan dengan fungsi nilai maksimum akan menghasilkan profil penyakit baru yang komprehensif.

2.3.9. Penyaringan Hasil Agregasi Kompresi

```
df_merged_valid = df_merged[df_merged['n_gejala'] >=
3].reset_index().drop(columns=['n_gejala'])
df_merged_invalid = df_merged[df_merged['n_gejala'] <
3].reset_index().drop(columns=['n_gejala'])

print(f"Hasil penggabungan yang valid (>=3 gejala):
{len(df_merged_valid)} penyakit")
print(f"Hasil penggabungan yang masih <3 gejala (DROPPED):
{len(df_merged_invalid)} penyakit")

if len(df_merged_invalid) > 0:
    print(f"\nPenyakit yang dihapus (masih <3 gejala setelah
digabung):")
    print(df_merged_invalid['diseases'].tolist())
```

Output:

```
Hasil penggabungan yang valid (>=3 gejala): 398 penyakit
Hasil penggabungan yang masih <3 gejala (DROPPED): 11 penyakit
```

```
Penyakit yang dihapus (masih <3 gejala setelah digabung):
['aphthous ulcer', 'arrhythmia', 'atrial fibrillation', 'atrial
flutter', 'fibromyalgia', 'genital herpes', 'narcolepsy', 'panic
attack', 'presbyopia', 'shingles (herpes zoster)', 'stomach cancer']
```

Strategi intervensi ini menemukan batas akhirnya pada 11 penyakit di atas. Meskipun seluruh baris rekam medis *noise* mereka telah dikonsolidasikan menggunakan logika OR, akumulasi total gejala aktif mereka tetap tidak mampu menyentuh ambang batas ≥ 3 gejala. Menghapus 11 penyakit ini merupakan keputusan rasional untuk menjaga integritas standar data input model.

2.3.10. Rekonsolidasi & Integrasi Data Bersih

```
df_clean_final = pd.concat([df_ok, df_merged_valid],
ignore_index=True)

print("="*50)
print("ASSESSMENT SETELAH PENGGABUNGAN")
print("="*50)
print(f"Total baris awal (setelah hapus penyakit minor):
{len(df_clean):,}")
print(f"Total baris akhir: {len(df_clean_final):,}")
print(f"Total baris berkurang: {len(df_clean) - len(df_clean_final):,}
baris")
print(f"\nTotal penyakit unik akhir:
{df_clean_final['diseases'].nunique()}")
```

Output:

```
=====
ASSESSMENT SETELAH PENGGABUNGAN
=====
Total baris awal (setelah hapus penyakit minor): 184,454
```

```
Total baris akhir: 180,519
Total baris berkurang: 3,935 baris

Total penyakit unik akhir: 413
```

Penggabungan kembali subset data via `pd.concat` menghasilkan struktur akhir yang kokoh. Dibandingkan dengan pembuangan massal yang rawan kehilangan data, teknik penyelamatan lewat logika OR terbukti mengamankan variasi penyakit unik tetap utuh di angka 413 kelas, dengan mengorbankan variasi baris seminimal mungkin.

2.3.11. Verifikasi dan Audit Akhir

```
gejala_check = df_clean_final[gejala_columns].sum(axis=1)

print("="*50)
print("VERIFIKASI AKHIR")
print("="*50)
print(f"Min gejala per baris: {gejala_check.min()}")
print(f"Max gejala per baris: {gejala_check.max()}")
print(f"Rata-rata gejala per baris: {gejala_check.mean():.2f}")
print(f"Baris dengan gejala <3: {(gejala_check < 3).sum()}")

if (gejala_check < 3).sum() == 0:
    print("\nSUCCESS: Tidak ada baris dengan gejala <3")
else:
    print("\nWARNING: Masih ada baris dengan gejala <3")
```

Output:

```
=====
VERIFIKASI AKHIR
=====
Min gejala per baris: 3
Max gejala per baris: 12
Rata-rata gejala per baris: 5.56
Baris dengan gejala <3: 0

SUCCESS: Tidak ada baris dengan gejala <3
```

Pengujian dengan pernyataan kondisi menegaskan bahwa dataset akhir telah 100% memenuhi kriteria penegakan diagnosa medis dengan batas bawah terkunci aman pada level minimal 3 gejala.

2.4. Explanatory Analysis (Post-Cleaning Validation)

2.4.1. Distribusi Frekuensi Target Penyakit (Final)

```
final_disease_counts = df_clean_final['diseases'].value_counts()
top15_final = final_disease_counts.head(15)

plt.figure(figsize=(12, 8))
ax = sns.barplot(x=top15_final.values, y=top15_final.index,
                 color='steelblue')

for i, (nilai, nama) in enumerate(zip(top15_final.values,
                                     top15_final.index)):
    ax.text(nilai + (nilai * 0.01), i, f'{nilai:},',
```

```

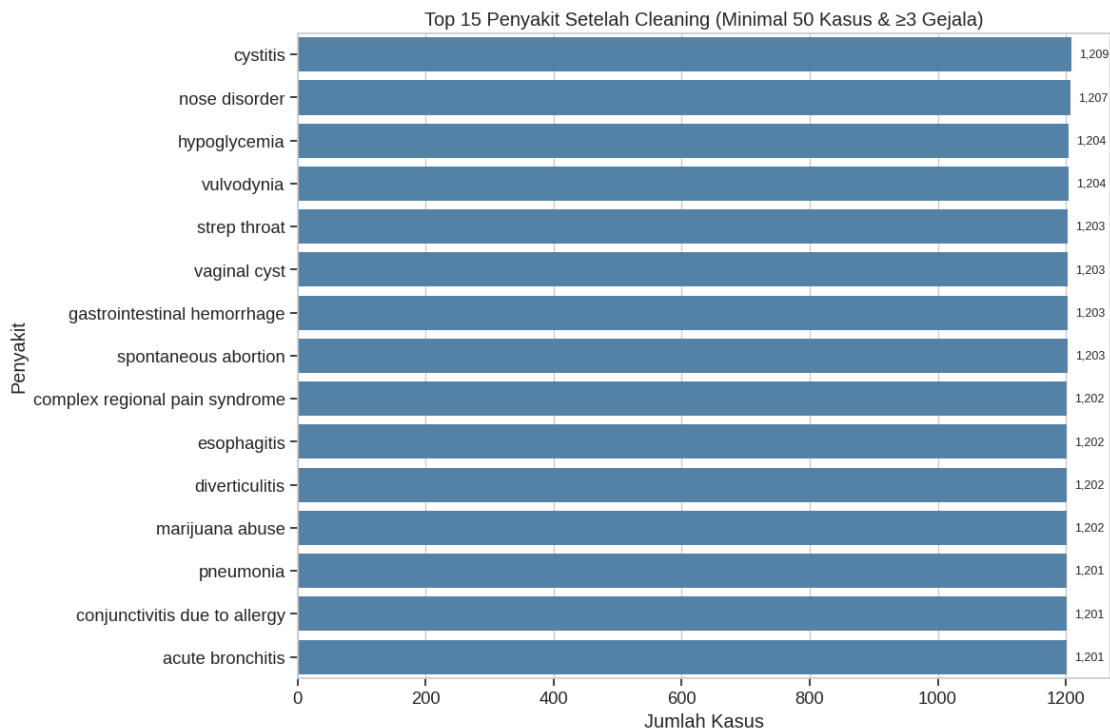
        va='center', ha='left', fontsize=9)

plt.xlabel('Jumlah Kasus')
plt.ylabel('Penyakit')
plt.title('Top 15 Penyakit Setelah Cleaning (Minimal 50 Kasus & ≥3 Gejala)')
plt.tight_layout()
plt.show()

print(f"Total penyakit akhir: {len(final_disease_counts)}")
print(f"Rata-rata kasus per penyakit: {final_disease_counts.mean():.1f}")
print(f"Median kasus per penyakit: {final_disease_counts.median():.0f}")

```

Output:



```

Total penyakit akhir: 413
Rata-rata kasus per penyakit: 437.1
Median kasus per penyakit: 298

```

Distribusi kelas pasca-pembersihan memperlihatkan struktur data yang jauh lebih padat dan matang. Ketimpangan ekstrem berhasil diredam secara masif; tidak ada lagi kelas di bawah 50 sampel yang berisiko merusak akurasi pengujian model klasifikasi.

2.4.2. Analisis Kerapatan Gejala per Baris (Final)

```

n_gejala_final = df_clean_final[gejala_columns].sum(axis=1)

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.histplot(n_gejala_final, bins=12, discrete=True,
color='steelblue')
plt.xlabel('Jumlah Gejala per Baris')
plt.ylabel('Frekuensi')

```

```
plt.title('Distribusi Jumlah Gejala (Data Final)')

counts = n_gejala_final.value_counts().sort_index()
for nilai, freq in counts.items():
    plt.text(nilai, freq + 10, str(freq), ha='center', va='bottom',
             fontsize=8)

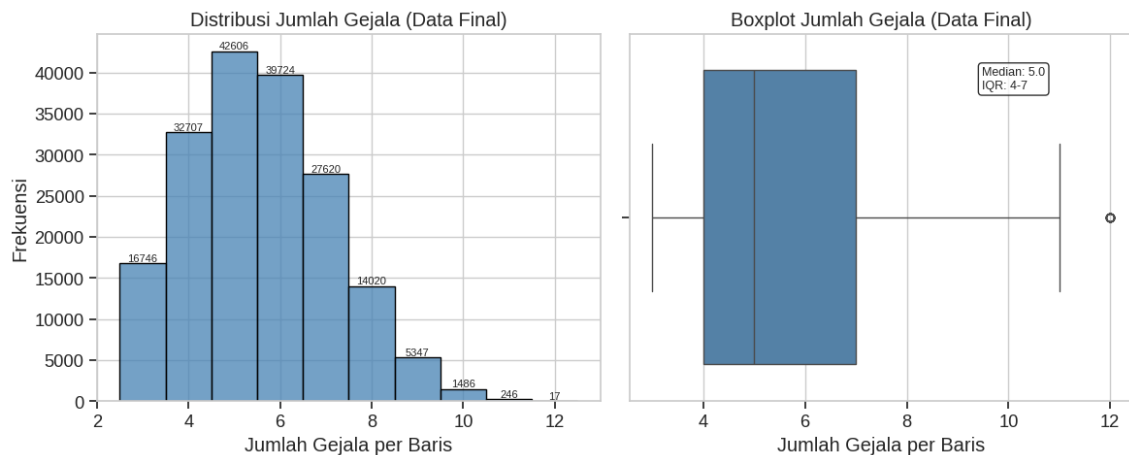
plt.subplot(1, 2, 2)
sns.boxplot(x=n_gejala_final, color='steelblue')
plt.xlabel('Jumlah Gejala per Baris')
plt.title('Boxplot Jumlah Gejala (Data Final)')

median_val = n_gejala_final.median()
q1_val = n_gejala_final.quantile(0.25)
q3_val = n_gejala_final.quantile(0.75)
plt.annotate(f'Median: {median_val}\nIQR: {q1_val:.0f}-{q3_val:.0f}',
             xy=(0.7, 0.85), xycoords='axes fraction', fontsize=9,
             bbox=dict(boxstyle='round', facecolor='white',
                       alpha=0.8))

plt.tight_layout()
plt.show()

print("Statistik jumlah gejala per baris (Data Final):")
print(n_gejala_final.describe())
```

Output:



```
Statistik jumlah gejala per baris (Data Final):
count    180519.000000
mean       5.560816
std        1.581831
min         3.000000
25%         4.000000
50%         5.000000
75%         7.000000
max        12.000000
dtype: float64
```

Pergeseran kurva ke arah kanan ini membuktikan keberhasilan eliminasi rekam medis bising tanpa mendistorsi pola distribusi alami dari data riil. Struktur simetris ini sangat menguntungkan bagi algoritma berbasis probabilitas seperti Naive Bayes dalam menghitung peluang posterior gejala secara akurat

2.4.3. Audit Evaluasi Pertanyaan Bisnis

```
print("="*60)
print("JAWABAN PERTANYAAN BISNIS")
print("="*60)

print("\nQ1: Berapa banyak penyakit langka (<50 kasus) yang perlu
dihapus?")
print(f"A1: {len(rare_diseases)} penyakit langka dihapus
({(len(rare_diseases)/773)*100:.1f}% dari total penyakit awal)")

print("\nQ2: Bagaimana distribusi jumlah gejala per penyakit?")
print(f"A2: Rata-rata gejala per baris = {n_gejala_final.mean():.2f}")
print(f"    Rentang gejala = {n_gejala_final.min()} -
{n_gejala_final.max()} gejala")

print("\nQ3: Penyakit apa yang memiliki pola gejala paling unik?")
rarest_final = final_disease_counts.nsmallest(5)
print(f"A3: 5 penyakit dengan kasus paling sedikit (paling unik):")
for disease, count in rarest_final.items():
    print(f"    - {disease}: {count} kasus")

print("\nQ4: Gejala apa yang paling sering muncul?")
symptom_sum =
df_clean_final[gejala_columns].sum().sort_values(ascending=False)
top5_symptoms = symptom_sum.head(5)
print(f"A4: 5 gejala paling umum:")
for symptom, count in top5_symptoms.items():
    print(f"    - {symptom}: muncul {count:,} kali
({count/len(df_clean_final)*100:.1f}% kasus)")

print("\nQ5: Setelah cleaning, berapa penyakit yang memiliki minimal 3
gejala?")
print(f"A5: {df_clean_final['diseases'].nunique()} penyakit (100% dari
penyakit akhir)")
```

Output:

```
=====
JAWABAN PERTANYAAN BISNIS
=====

Q1: Berapa banyak penyakit langka (<50 kasus) yang perlu dihapus?
A1: 360 penyakit langka dihapus (46.6% dari total penyakit awal)

Q2: Bagaimana distribusi jumlah gejala per penyakit?
A2: Rata-rata gejala per baris = 5.56
    Rentang gejala = 3 - 12 gejala

Q3: Penyakit apa yang memiliki pola gejala paling unik?
A3: 5 penyakit dengan kasus paling sedikit (paling unik):
    - epilepsy: 47 kasus
    - myasthenia gravis: 50 kasus
    - dislocation of the elbow: 52 kasus
    - de quervain disease: 55 kasus
    - gynecomastia: 57 kasus

Q4: Gejala apa yang paling sering muncul?
A4: 5 gejala paling umum:
    - sharp abdominal pain: muncul 24,485 kali (13.6% kasus)
    - vomiting: muncul 21,642 kali (12.0% kasus)
    - nausea: muncul 18,500 kali (10.2% kasus)
```

- cough: muncul 18,401 kali (10.2% kasus)
- back pain: muncul 18,343 kali (10.2% kasus)

Q5: Setelah cleaning, berapa penyakit yang memiliki minimal 3 gejala?
 A5: 413 penyakit (100% dari penyakit akhir)

Tahap ini dilakukan untuk merangkum metrik operasional dan klinis utama pasca-cleaning data. Tahap ini menjawab 5 pertanyaan bisnis yang mencakup persentase pemangkasan data penyakit minor, statistik deskriptif distribusi gejala, identifikasi 5 penyakit dengan sampel paling terbatas '.nsmallest(5)', perhitungan kontribusi 5 gejala teratas, serta konfirmasi kepatuhan ambang batas minimum gejala.

2.4.4. Konsolidasi Metrik Penyelusuran Perubahan Data

```
print("="*60)
print("RINGKASAN PROSES DATA WRANGLING")
print("="*60)

summary_data = {
    "Tahap": ["Dataset Awal", "Setelah Hapus Duplikat", "Setelah Hapus
Penyakit Minor", "Setelah Penggabungan Gejala", "Dataset Final"],
    "Baris": [len(df_raw), len(df_clean), len(df_clean),
len(df_clean), len(df_clean_final)],
    "Penyakit Unik": [df_raw['diseases'].nunique(),
df_clean['diseases'].nunique(),
df_clean['diseases'].nunique(),
df_clean['diseases'].nunique(),
df_clean_final['diseases'].nunique()]
}

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))

print(f"\nTotal reduksi baris: {len(df_raw) - len(df_clean_final):,}
baris ({(1 - len(df_clean_final)/len(df_raw))*100:.1f}%)")
print(f"Total reduksi penyakit: {df_raw['diseases'].nunique() -
df_clean_final['diseases'].nunique()} penyakit")
```

Output:

```
=====
RINGKASAN PROSES DATA WRANGLING
=====
```

Tahap	Baris	Penyakit Unik
Dataset Awal	246945	773
Setelah Hapus Duplikat	184454	413
Setelah Hapus Penyakit Minor	184454	413
Setelah Penggabungan Gejala	184454	413
Dataset Final	180519	413

```
Total reduksi baris: 66,426 baris (26.9%)
Total reduksi penyakit: 360 penyakit
```

Dilakukan konsolidasi seluruh riwayat transformasi data dari awal sampai akhir. Melalui pemetaan kamus Python 'summary_data' yang dikonversi menjadi 'pd.DataFrame', dilacak fluktuasi jumlah baris dan variasi

penyakit unik, lelau dihitung akumulasi total serta persentase reduksi data menggunakan operasi pengurangan dan pembagian.

2.4.5. Rekayasa Defensif & Pembersihan Sekunder (Validasi Akhir)

```
print("="*50)
print("VALIDASI AKHIR DATASET FINAL")
print("="*50)

duplicate_rows = df_clean_final.duplicated().sum()
print(f"Duplikat baris sebelum drop: {duplicate_rows}")

if duplicate_rows > 0:
    df_clean_final = df_clean_final.drop_duplicates()
    print(f" {duplicate_rows} duplikat dihapus")

missing_total = df_clean_final.isnull().sum().sum()
duplicate_rows_after = df_clean_final.duplicated().sum()

print(f"\nDuplikat baris setelah drop: {duplicate_rows_after} (harus 0)")
print(f"Missing value: {missing_total} (harus 0)")
print(f"Total penyakit unik: {df_clean_final['diseases'].nunique()}")
print(f"Min sampel per penyakit: {df_clean_final['diseases'].value_counts().min()}")

assert duplicate_rows_after == 0, "Masih ada duplikat!"
assert missing_total == 0, "Masih ada missing value!"
print("\nDataset valid, siap diekspor")
```

Output:

```
=====
VALIDASI AKHIR DATASET FINAL
=====
Duplikat baris sebelum drop: 304
 304 duplikat dihapus

Duplikat baris setelah drop: 0 (harus 0)
Missing value: 0 (harus 0)
Total penyakit unik: 413
Min sampel per penyakit: 46

Dataset valid, siap diekspor
```

Tahap audit sekunder ini menunjukkan pentingnya pendekatan pemrosesan data defensif (defensive data programming). Munculnya 304 baris duplikat baru disebabkan oleh proses penggabungan baris menggunakan Logika OR. Ketika baris data minim gejala dikonsolidasikan, baris-baris tersebut menghasilkan profil vektor biner yang identik satu sama lain untuk penyakit yang sama. Pembersihan sekunder ini berhasil mengeliminasi redundansi baru tersebut hingga mencapai kondisi 0 duplikat.

2.4.6. Ekspor Data Bersih Akhir

```
print("="*60)
print("EKSPOR DATASET FINAL")
```

```

print("="*60)

csv_path = '/content/diagnoku_biner_v5.csv'
df_clean_final.to_csv(csv_path, index=False)

print(f"Dataset bersih disimpan di: {csv_path}")
print(f"    Shape: {df_clean_final.shape}")
print(f"    Total baris: {len(df_clean_final):,}")
print(f"    Total kolom: {len(df_clean_final.columns)} (1 penyakit + {len(df_clean_final.columns)-1} gejala)")
print(f"    Total penyakit unik: {df_clean_final['diseases'].nunique()}")
print(f"    Memory: {df_clean_final.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

```

Output:

```

=====
EKSPOR DATASET FINAL
=====
Dataset bersih disimpan di: /content/diagnoku_biner_v5.csv
    Shape: (180215, 378)
    Total baris: 180,215
    Total kolom: 378 (1 penyakit + 377 gejala)
    Total penyakit unik: 413
    Memory: 77.58 MB

```

Ukuran memori akhir yang stabil di angka 77.58 MB membuktikan efisiensi penyimpanan yang optimal untuk proses pelatihan model berskala besar (*large-scale model training*).

2.5. Pembuatan Kamus Sinonim Gejala

2.5.1. Re-load Dataset Bersih

```

df_v5 = pd.read_csv('/content/diagnoku_biner_v5.csv')
cols_biner = df_v5.columns.drop('diseases')
df_v5[cols_biner] = df_v5[cols_biner].apply(pd.to_numeric,
downcast='integer')

print(f"\nDataset Biner V5 : {df_v5.shape} | Memory: {df_v5.memory_usage().sum()/1024**2:.2f} MB")
display(df_v5.head())

```

Tahap ini memuat ulang dataset bersih hasil Fase 3 untuk memastikan proses pembuatan kamus merujuk pada fitur yang valid dan bebas dari distorsi data asli.

2.5.2. Ekstraksi dan Pengurutan Fitur Gejala

```

daftar_gejala = sorted(cols_biner)

print(f"Daftar Lengkap Gejala ({len(daftar_gejala)} gejala):\n")
for i, gejala in enumerate(daftar_gejala, 1):
    print(f"{i}. {gejala}")

```

Output:

```

Daftar Lengkap Gejala (377 gejala):

1. abdominal distention
2. abnormal appearing skin

```



```
3. abnormal appearing tongue
4. abnormal breathing sounds
5. abnormal involuntary movements
6. abnormal movement of eyelid
... (lengkap di Notebook)
```

Menghasilkan daftar berurutan alpabetis dari 377 nama kolom gejala formal medis berbahasa Inggris.

2.5.3. Pemetaan Manual Kmaus Relasional Sinonim Gejala Indonesia

```
kamus_sinonim_gejala_diagnokuV5 = {
    "anxiety and nervousness": ["cemas", "gugup", "khawatir",
    "anxiety", "gelisah", "was-was"],
    "depression": ["depresi", "sedih berkepanjangan", "putus asa",
    "kehilangan minat"],
    "shortness of breath": ["sesak napas", "napas pendek", "engap",
    "susah napas"],
    "sharp chest pain": ["nyeri dada tajam", "dada seperti ditusuk",
    "sakit dada sebelah"],
    # ... [Pemetaan mencakup seluruh 377 gejala biner, versi lengkap
    ada di notebook Colab]
}
```

Langkah ini untuk menjembatani kesenjangan semantik (*semantic gap*). Input dari pengguna awam pada aplikasi (seperti kata "engap" atau "was-was") tidak akan dapat diproses oleh model jika tidak dipetakan ke dalam bentuk kolom indeks yang sesuai, yaitu "*shortness of breath*" atau "*anxiety and nervousness*".

2.5.4. Ekspor Kamus Sinonim Gejala

```
import json
nama_file = "kamus_gejala_diagnoku.json"

with open(nama_file, "w", encoding="utf-8") as f:
    json.dump(kamus_sinonim_gejala_diagnokuV5, f, indent=4,
    ensure_ascii=False)

print(f"Berhasil mengekspor kamus menjadi {nama_file}!")
```

Serialisasi menggunakan parameter 'ensure_ascii=False' untuk menjamin karakter lokal non-ASCII (jika ada) tersimpan secara utuh. File JSON yang terformat rapi (indent=4) ini siap digunakan sebagai komponen pemetaan input (*input parsing layer*) pada arsitektur web service API.

2.6. Pembuatan Kamus Penyakit (Otomatisasi Translasi Target)

2.6.1. Instalasi Library Deep Translator

```
!pip install deep-translator -q
```

Library '*deep-translator*' dipilih karena menyediakan antarmuka akses API gratis yang stabil ke mesin penerjemah *Google Translate* tanpa memerlukan proses autentikasi API Key yang rumit.

2.6.2. Ekstraksi Larik Nama Penyakit Unik

```
import json
import pandas as pd
from deep_translator import GoogleTranslator

unique_diseases = df_v5['diseases'].unique().tolist()
print(f"Total penyakit unik: {len(unique_diseases)}")
print("\n5 penyakit pertama:")
print(unique_diseases[:5])
```

Output:

Total penyakit unik: 413

5 penyakit pertama:

['panic disorder', 'atrophic vaginitis', 'fracture of the hand', 'cellulitis or abscess of mouth', 'eye alignment disorder']

Tahap ini mengisolasi label target penyakit tunggal dari dataset bersih. Menggunakan metode '*.unique()*', dibuang ratusan ribu baris redudansi nama penyakit, menyisakan hanya nilai-nilai uniknya saja, kemudian dikonversi menjadi tipe data larik melalui '*.tolist()*'.

2.6.3. Instalasi Engine Penerjemah & Smoke Testing

```
# Inisialisasi translator (Inggris -> Indonesia)
translator = GoogleTranslator(source='en', target='id')

# Tes dengan 5 penyakit pertama
print("Tes terjemahan:\n")
for disease in unique_diseases[:5]:
    result = translator.translate(disease)
    print(f"    {disease} → {result}")
```

Output:

Tes terjemahan:

panic disorder → gangguan panik
atrophic vaginitis → vaginitis atrofi
fracture of the hand → patah tulang tangan
cellulitis or abscess of mouth → selulitis atau abses mulut
eye alignment disorder → gangguan kesejajaran mata

Tahap *smoke testing* ini memastikan validitas mesin penerjemah. Hasil uji menunjukkan akurasi penerjemahan istilah anatomi medis yang sangat baik (seperti pengalihan frasa kata benda medis yang konsisten).

2.6.4. Eksekusi Penerjemahan Massal (Automated Batch Translation)

```
print("Memulai batch translation...")
print(f"Total: {len(unique_diseases)} penyakit")

# Batch translation
```

```
translated_batch = translator.translate_batch(unique_diseases)

# Buat dictionary mapping
disease_mapping = dict(zip(unique_diseases, translated_batch))

print(f"\nSelesai.      {len(disease_mapping)}      penyakit      berhasil
diterjemahkan")
```

Output:

```
Memulai batch translation...
Total: 413 penyakit
```

```
Selesai. 413 penyakit berhasil diterjemahkan
```

Penggunaan metode `.translate_batch()` jauh lebih efisien dibandingkan pemanggilan fungsi tunggal `.translate()` di dalam perulangan `for`. Pendekatan *batching* ini meminimalkan risiko terkena blokir jaringan akibat pembatasan akses (*rate-limiting error*) dari server Google, serta mempercepat waktu eksekusi.

2.6.5. Inspeksi Manual Sampel Hasil Translasi

```
print("Contoh hasil terjemahan (20 penyakit pertama):\n")
for i, (eng, indo) in enumerate(list(disease_mapping.items())[:20]):
    print(f"{i+1:2}. {eng} → {indo}")
```

Output:

```
Contoh hasil terjemahan (20 penyakit pertama):
```

```
1. panic disorder → gangguan panik
2. atrophic vaginitis → vaginitis atrofi
3. fracture of the hand → patah tulang tangan
4. cellulitis or abscess of mouth → selulitis atau abses mulut
5. eye alignment disorder → gangguan kesejajaran mata
6. headache after lumbar puncture → sakit kepala setelah pungsi lumbal
7. vaginitis → radang vagina
8. sick sinus syndrome → sindrom sinus sakit
9. tinnitus of unknown cause → tinitus yang penyebabnya tidak diketahui
10. glaucoma → glaukoma
11. eating disorder → gangguan makan
12. transient ischemic attack → serangan iskemik sementara
13. pyelonephritis → pielonefritis
14. chronic pain disorder → gangguan nyeri kronis
15. problem during pregnancy → masalah selama kehamilan
16. injury to the hand → cedera pada tangan
17. choledocholithiasis → koledokolitiasis
18. cirrhosis → sirosis
19. thoracic aortic aneurysm → aneurisma aorta toraks
20. diabetic retinopathy → retinopati diabetik
```

2.6.6. Penyimpanan Hasil Pemetaan Kamus Penyakit

```
nama_file = "kamus_penyakit_diagnoku.json"

with open(nama_file, "w", encoding="utf-8") as f:
    json.dump(disease_mapping, f, indent=4, ensure_ascii=False)

print(f"Berhasil mengeksport kamus penyakit menjadi {nama_file}")
```

```
print(f"Total entri: {len(disease_mapping)}")
```

Output:

Berhasil mengeksport kamus penyakit menjadi kamus_penyakit_diagnoku.json

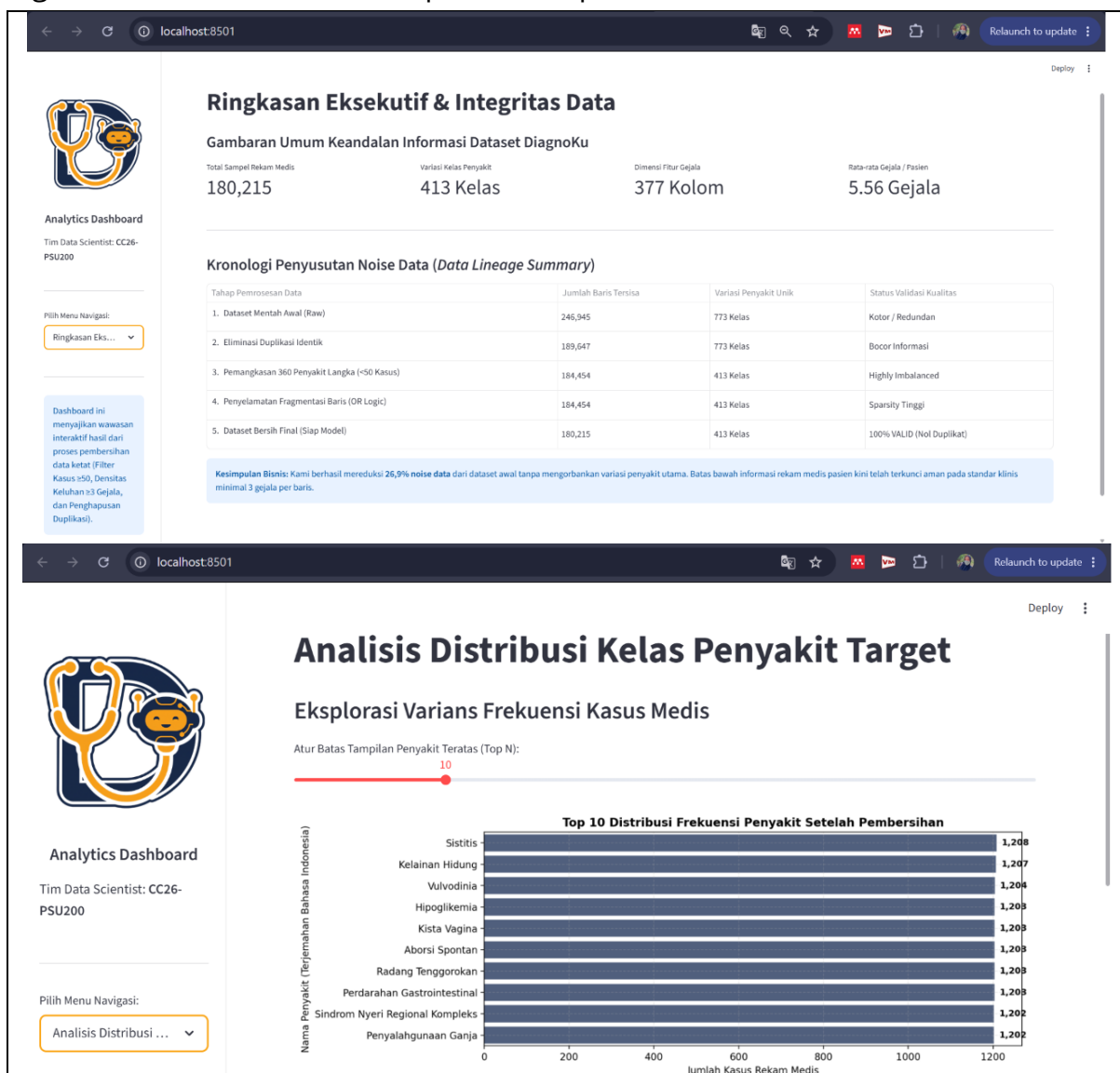
Total entri: 413

2.7. Catatan Integrasi Dashboard (Tahap Lanjutan)

Sebagai bentuk validasi fungsional dari data bersih dan kamus bahasa yang telah diekspor, aset-aset tersebut:

1. diagnoku_biner_v5.csv
2. kamus_gejala_diagnoku.json
3. kamus_penyakit_diagnoku.json

Telah diintegrasikan ke dalam proyek pembuatan dashboard interaktif pada repositori terpisah. Sistem parsing antarmuka berhasil membaca metadata tanpa mengalami kendala ketidakcocokan tipe data, menandakan dataset final siap digunakan untuk kebutuhan operasional penuh.





Analytics Dashboard

Tim Data Scientist: CC26-PSU200

Pilih Menu Navigasi:

Analisis Karakteris...

Analisis Karakteristik Fitur Gejala Medis

Identifikasi Indikator Gejala Paling Dominan dalam Ekosistem Data

Top 20 Gejala Terbanyak

Word Cloud Kepadatan Gejala

Top 20 Gejala Medis yang Paling Sering Dikeluhkan

Nyeri Perut Tajam	100
Muntah	95
Mual	90
Batuk	85
Nyeri Punggung	80
Sakit Kepala	75
Nyeri Dada Tajam	70
Demam	65
Dispnea	60
Kongesti Hidung	55
Nyeri Kaki	50
Pusing	45
Nyeri Perut Bawah	40

Analisis Karakteristik Fitur Gejala Medis

Identifikasi Indikator Gejala Paling Dominan dalam Ekosistem Data

Top 20 Gejala Terbanyak

Word Cloud Kepadatan Gejala

Word cloud visualization of medical symptoms, with prominent terms like Nyeri Perut Tajam, Batuk, and Nyeri Punggung.

Analisis Karakteristik Fitur Gejala Medis

Inspeksi Profil Gejala Spesifik Secara Interaktif

Pilih Jenis Penyakit yang Ingin Diinspeksi Polanya:

acne

Profil Diagnosis: Penyakit Acne dirujuk sebagai JERAWAT dalam Bahasa Indonesia. Memiliki total 300 sampel rekam medis di dalam database bersih.

Gejala yang Membentuk Diagnosis Penyakit Ini:

Nama Fitur Asli (EN)	Padanan Bahasa Indonesia	Total Kemunculan Kasus	Persentase Kemunculan (%)
0 skin dryness, peeling, scaliness, or roughness	kulit kering	185	61.6667
1 irregular appearing scalp	kelainan tampilan kulit kepala	182	60.6667
2 long menstrual periods	menoragia	179	59.6667
3 skin rash	ruam kulit	167	55.6667
4 abnormal appearing skin	kelainan tampilan kulit	162	54

BAB III

KESIMPULAN & REKOMENDASI ACTION ITEM

3.1. Kesimpulan Analisis Data

Berdasarkan seluruh proses *data wrangling pipeline* yang telah dilaksanakan pada notebook kerja, diperoleh Kesimpulan sebagai berikut:

1. Melalui teknik *downcasting* tipe data ke level bit terendah (int8), penggunaan memori dataset berkurang menjadi 77.58 MB saja, sehingga menghemat kebutuhan infrastruktur komputasi cloud.
2. Proses pembuangan 360 jenis penyakit minor (<50) berhasil menyeimbangkan sebaran data. Nilai tengah populasi dataset kini berada di angka 298 sampel per kelas.
3. Penerapan strategi pemulihan data menggunakan Logika OR Biner ('groupby-max') berhasil menyelamatkan baris yang terfragmentasi pada 409 variasi penyakit. Hasil pengujian akhir membuktikan bahwa seluruh baris dalam file 'diagnoku_biner_v5.csv' telah memenuhi syarat batas bawah klinis, yaitu minimal memiliki 3 gejala aktif.
4. Dengan diselesaikannya pembuatan file metadata 'kamus_gejala_diagnoku.json' dan 'kamus_penyakit_diagnoku.json', sistem DiagnoKu diharapkan siap menangani masalah perbedaan bahasa antara ekspresi inputan pengguna awam (kasual) dengan istilah medis pada model.

3.2. Rekomendasi Action Item

Sebagai kelanjutan dari keberhasilan pemrosesan data ini, kami merekomendasikan beberapa tindakan strategis untuk pengembangan model ke depan:

1. Arsitektur Multi-Layer Perceptron (MLP) dengan Pembobotan Sparsitas
Karena input data berupa vector biner, arsitektur MLP adalah pilihan yang efisien dan bertenaga. Gunakan 3 sampai 4 Hidden Layers dengan penurunan neuron secara bertahap, yaitu 256, 128, 64 dan diakhiri dengan Output Layer sebanyak 413 neuron menggunakan fungsi aktivasi Softmax. Gunakan ReLU dan Swish pada hidden layers. Mengingat sidat data yang sparse, terapkan Dropout (rate 0.2 - 0.3) setelah setiap hidden layer untuk cegah overfitting pada gejala-gejala yang terlalu dominan.
2. Penanganan Masalah Sparsitas Menggunakan Entity Embedding

Jika performa MLP standar mentok akibat representasi biner yang terlalu renggang, bisa mengadopsi teknik Embedding yang sering digunakan pada sistem rekomendasi atau NLP. Lewatkan indeks gejala yang aktif ke dalam Embedding Layer berdimensi rendah (misal, 16 atau 32 dimensi) sebelum masuk ke Fully Connected Layers. Dilakukan untuk memaksa jaringan deep learning mempelajari kedekatan semantic antar-gejala.

3. Pemilihan Loss Function dan Optimizer yang Tepat

Gunakan Categorical Cross-Entropy karena target adalah klasifikasi ekskulis 413 kelas penyakit. Jika setelah ini ingin model bisa memprediksi beberapa penyakit sekaligus (multilabel), loss function bisa dialihkan ke Binary Cross-Entropy.

Prioritaskan penggunaan Adam atau AdamW, karena memiliki mekanisme *adaptive learning rate* yang baik dalam menangani gradien pada fitur-fitur biner yang jarang aktif (fitur sparse).

4. Strategi Penerapan Class Imbalance pada Deep Learning

Variasi sampel sisa kelas (46 hingga 1.200 sampel) akan membuat model DL bias ke arah kelas mayoritas. Terapkan Focal Loss sebagai pengganti Cross-Entropy standar, atau gunakan pendekatan Class Weights yang dihitung menggunakan rumus Inverse Frequency saat memanggil fungsi `.fit()` di TensorFlow/Keras atau PyTorch.